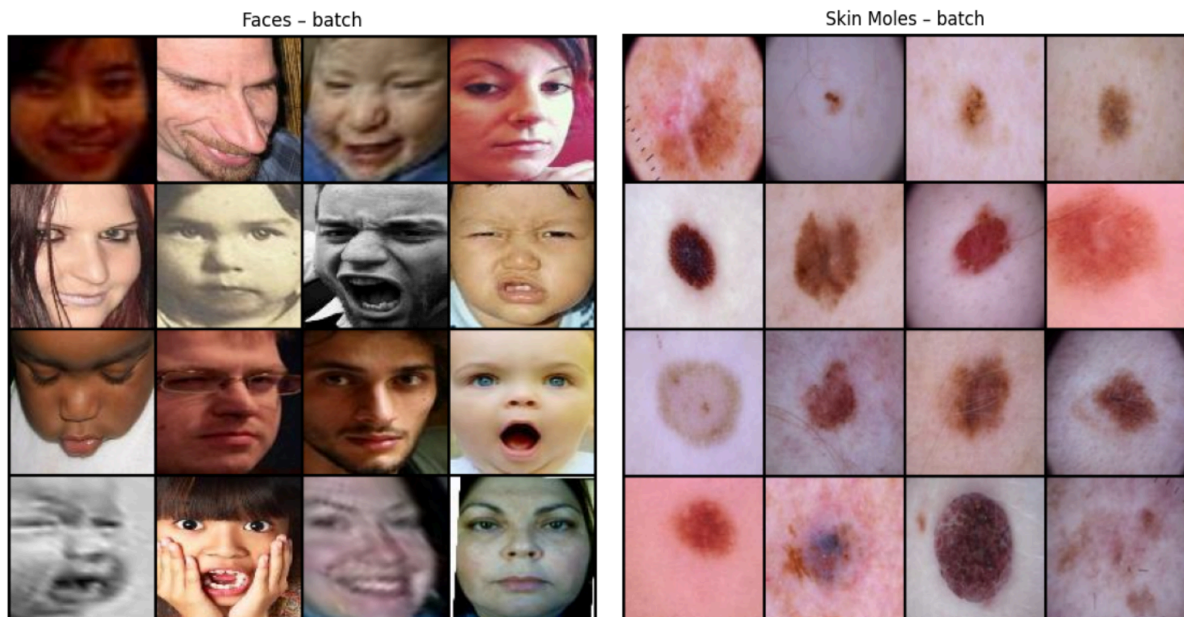


Tema 2

Jurnalizare

Pentru inceput, am creat dataseturile si dataloaderele. Am facut transformari minimale asupra datelor, transformari care ne vor servi ca baseline:

- Redimesnsionare - toate imaginile au devenit de 100 x 100
- Conversie in tensor - format pe care modelul sa il poata intelege (numere in loc de imagini)
- Normalizare (0 - 255)



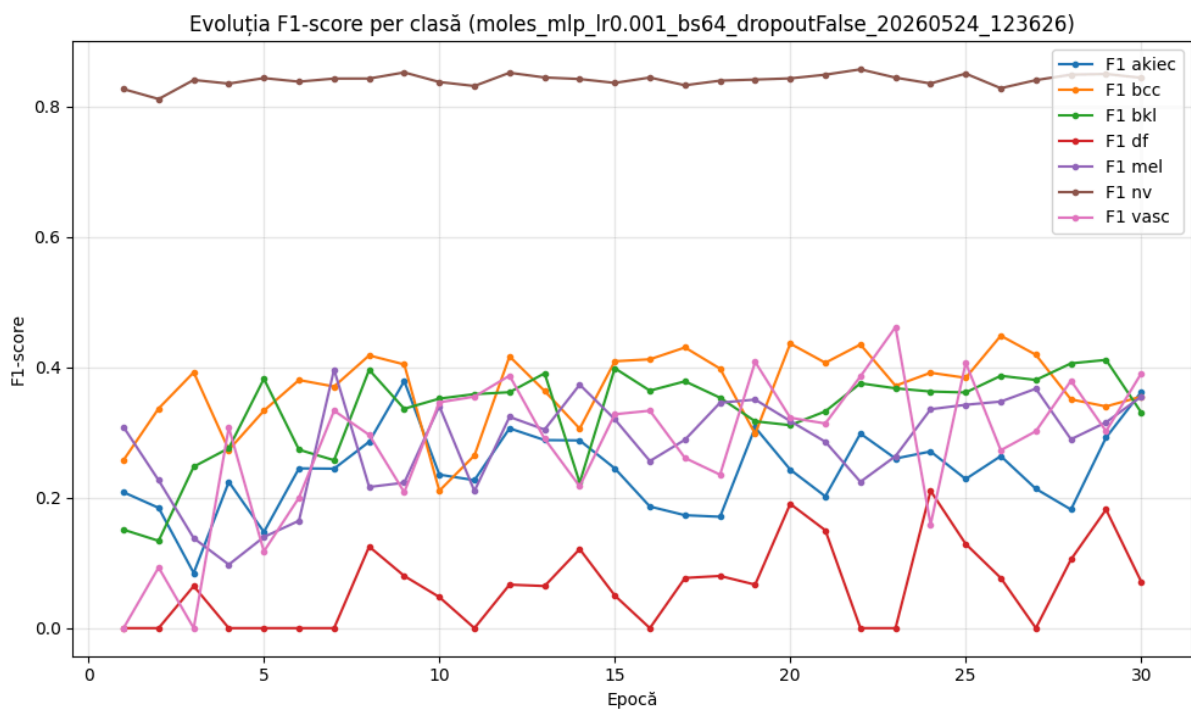
MOLES

Modelul MLP-Simplu (Fara Dropout):

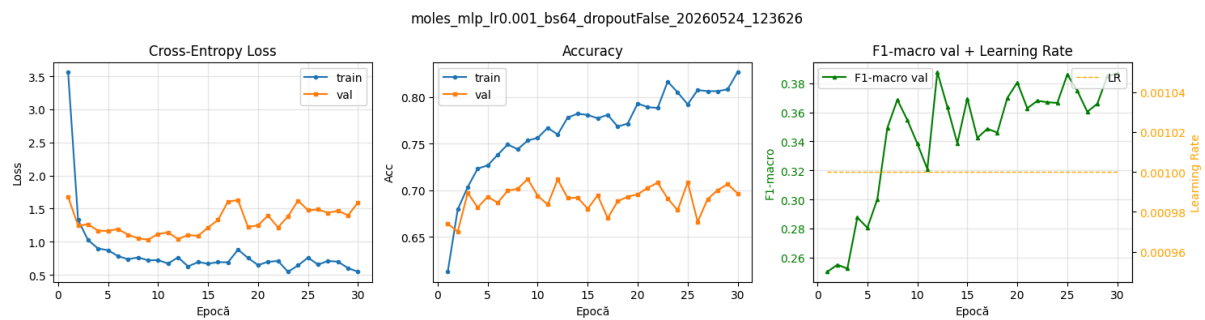
Am folosit Flatten pentru transformarea imaginilor in vectori, Linear la 512, ReLu (un filtru simplu) si iar Linear pt a colapsa rezultatele in procente pentru cele 7 clase.

```
critterion = nn.CrossEntropyLoss()  
LR = 1e-3  
optimizer = optim.Adam(model_moles.parameters(), lr=LR,  
weight_decay=1e-4)
```

Am antrenat 30 de epoci.

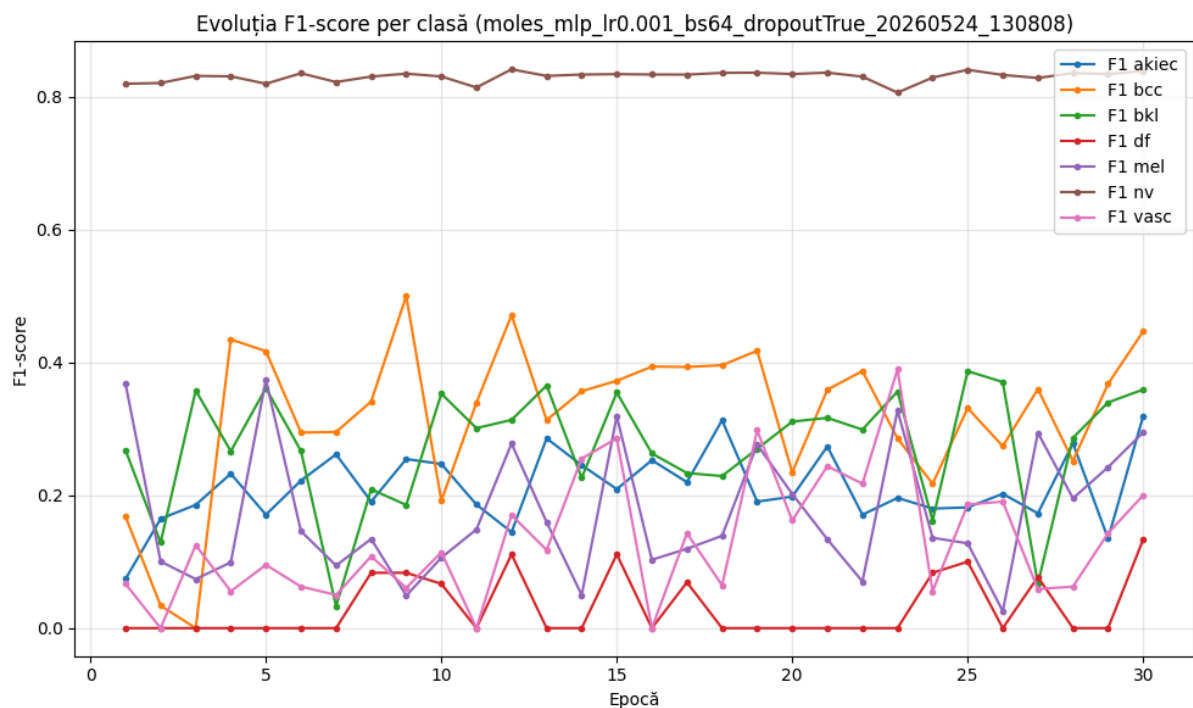


Se observa diferenta mare dintre scorurile F1 ale diferitelor clase, rezultatele fiind foarte proaste pentru clase minoritare precum df.

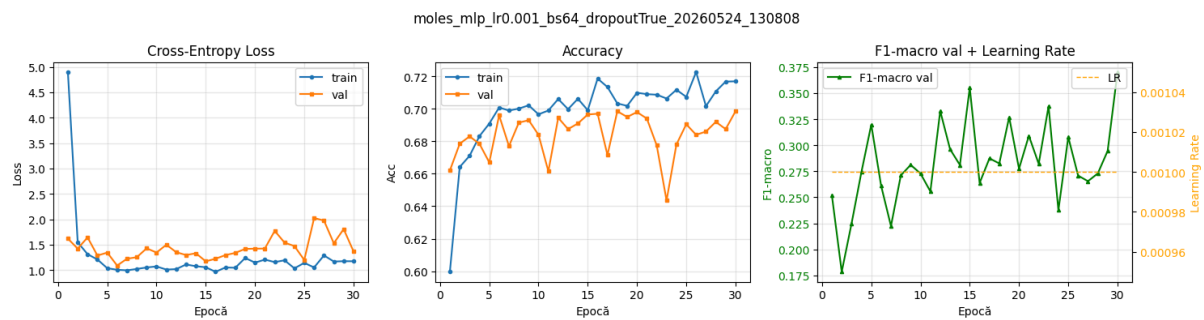


Modelul ajunge sa faca Overfitting pe datele de antrenament.
Pe teste, acuratetea obtinuta e in jur de 70% dar F1-macro
ajunge la maxim 0.38, mult prea putin.

Modelul MLP-Simplu (Cu Dropout):



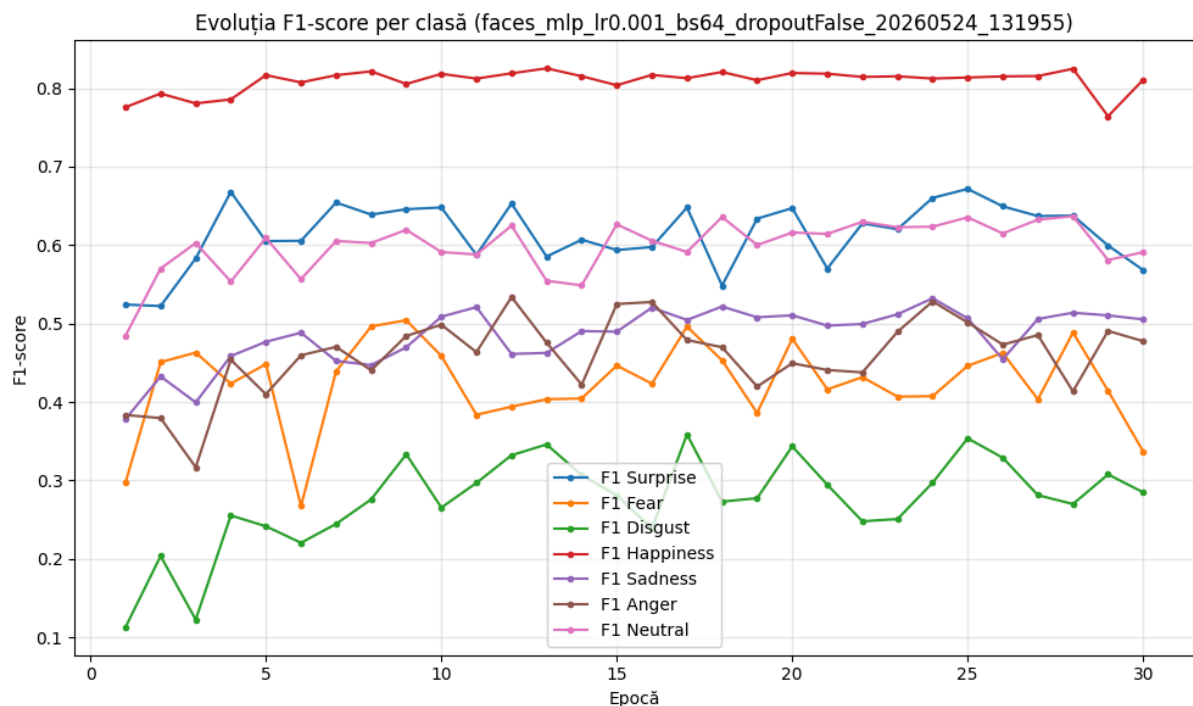
Nu se observa imbunatatiri.



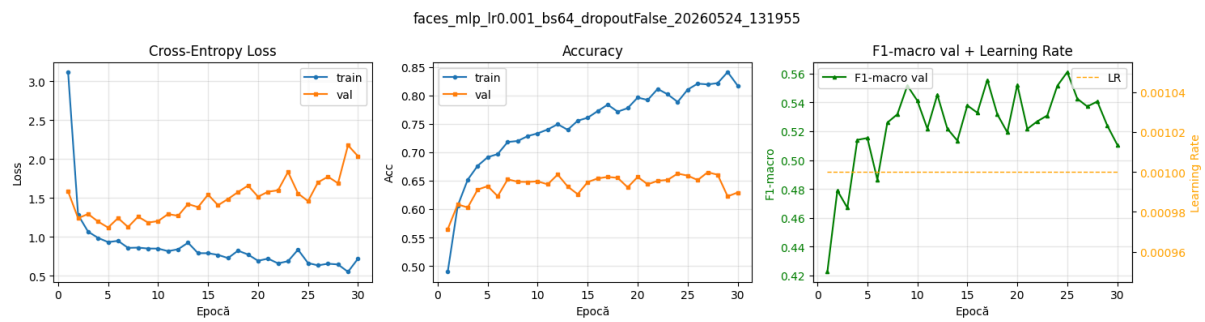
Modelul nu mai face asa de mult Overfitting, Dropoutul ajutandu-l sa imi imbunatateasca capacitatea de a generaliza. Totusi, rezultatele sunt in continuare slabe.

FACES

Modelul MLP-Simplu (Fara Dropout):

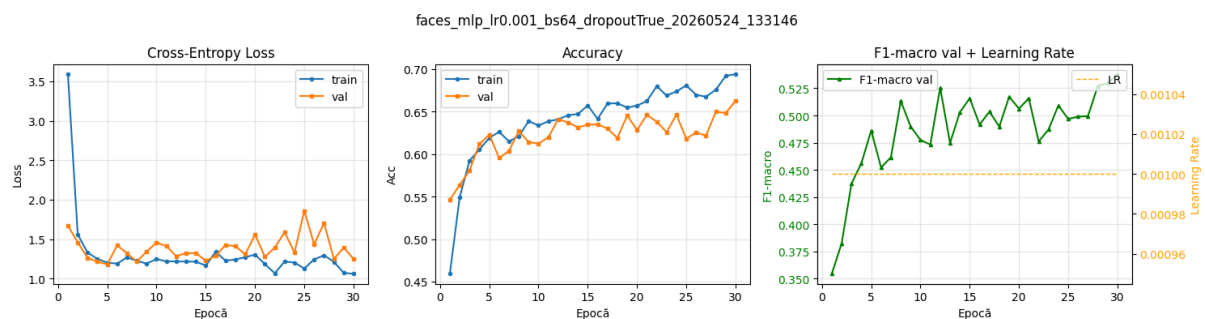
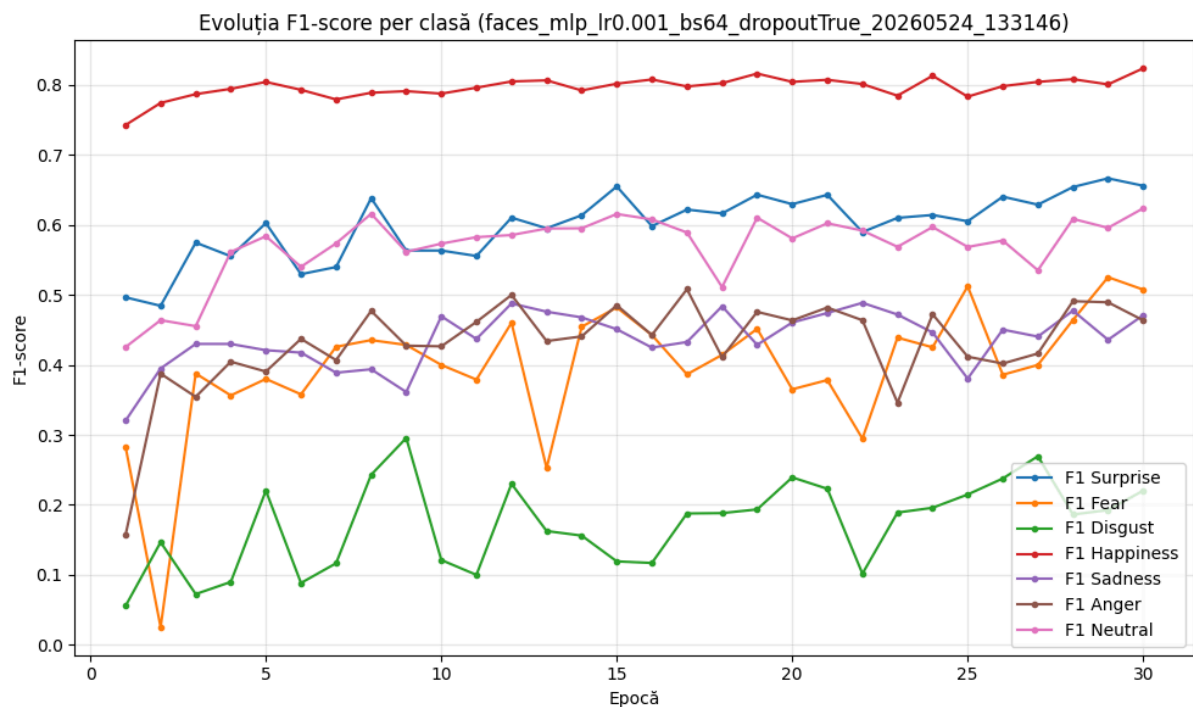


Clasele sunt putin mai echilibrate decat in setul Moles.



In rest, aceleasi observatii, overfitting.

Model Mlp-Simplu (Cu Dropout):



MOLES

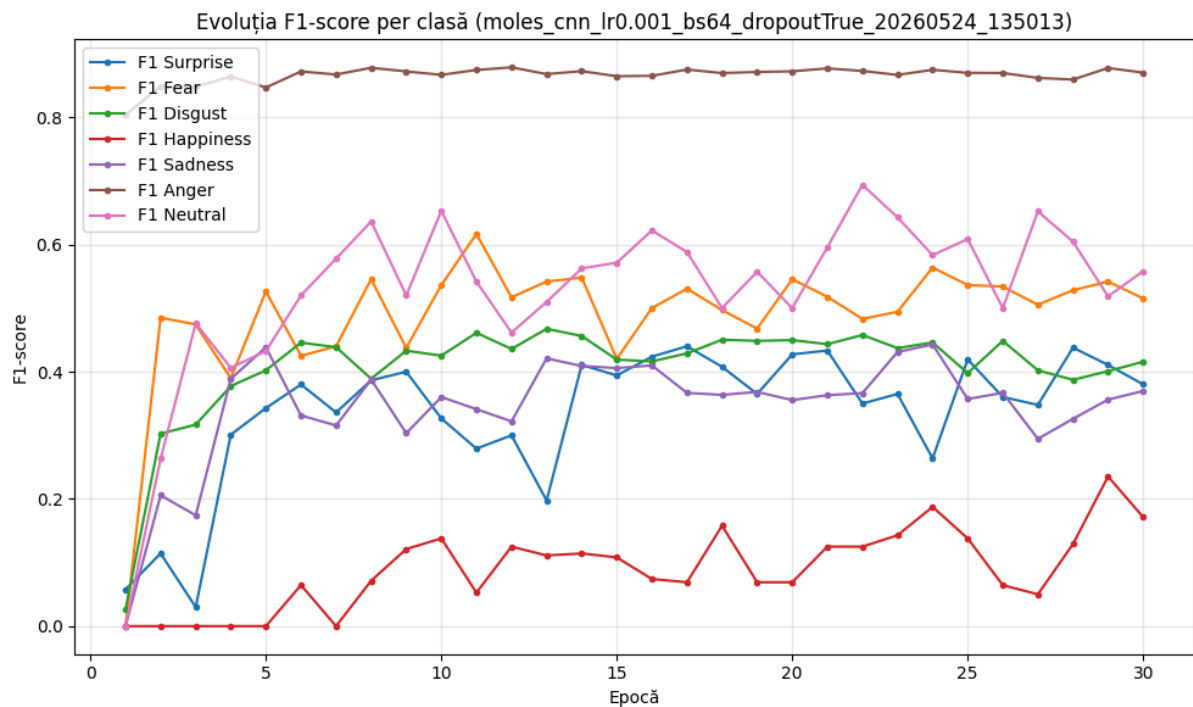
Modelul CNN-Simplu:

Am folosit Conv2d pentru a aplica filtre pe imagine, ReLu si MaxPool pentru a micsora imaginea.

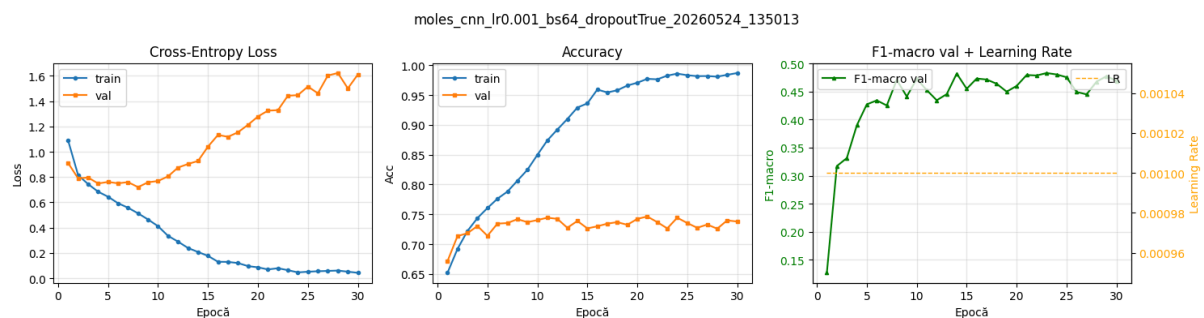
```
# Bloc 1
nn.Conv2d(input_channels, 32, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.MaxPool2d(2),

# Bloc 2
nn.Conv2d(32, 64, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.MaxPool2d(2),
)

feat_h = img_height // 4
feat_w = img_width // 4
input_dim = 64 * feat_h * feat_w
```



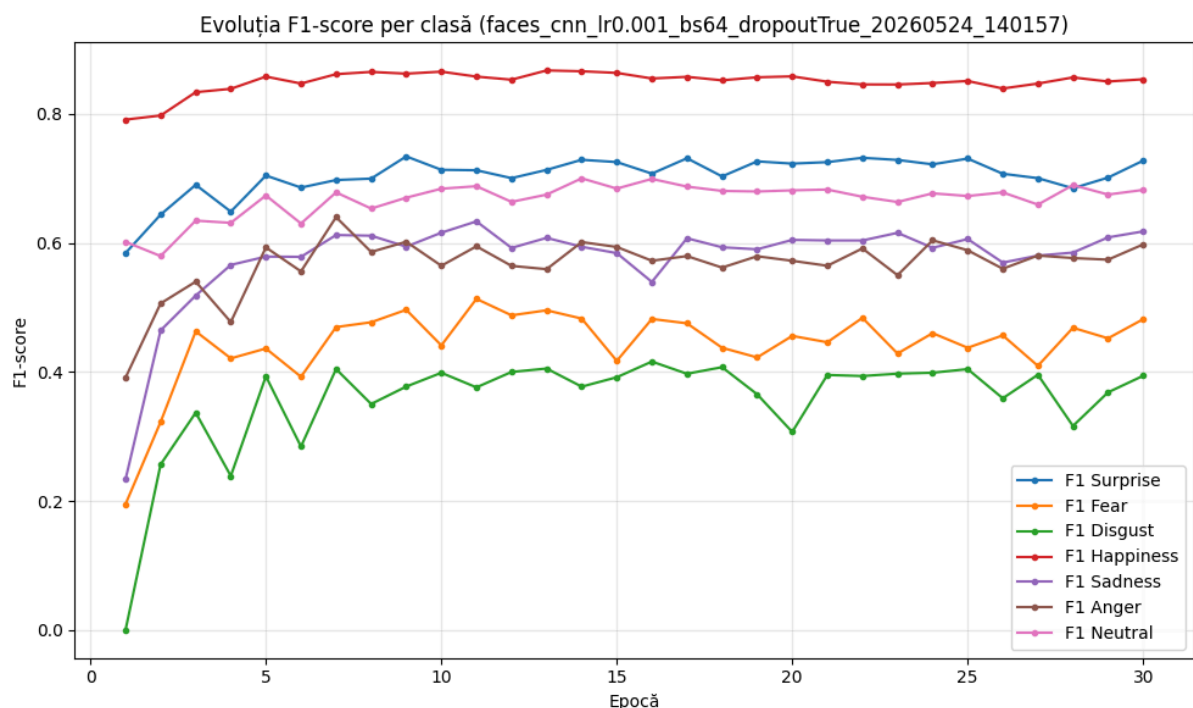
Se observa ca CNN-ul diferentiaza mai bine clasele decat MLP-ul F1-macro atinand maximul de 0.5 fata de o.375.



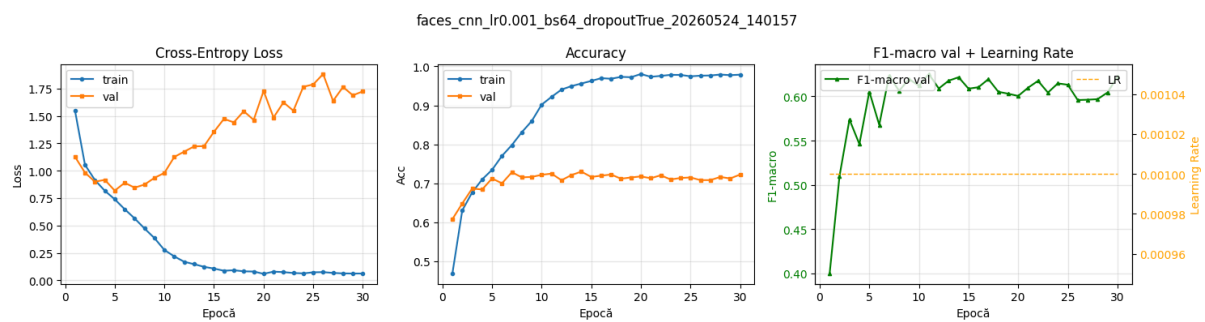
Acuratetea si Lossul ajung la valori perfecte pe training dar din cauza overfitting-ului nu se descurca pe teste.

FACES

Modelul CNN-Simplu:



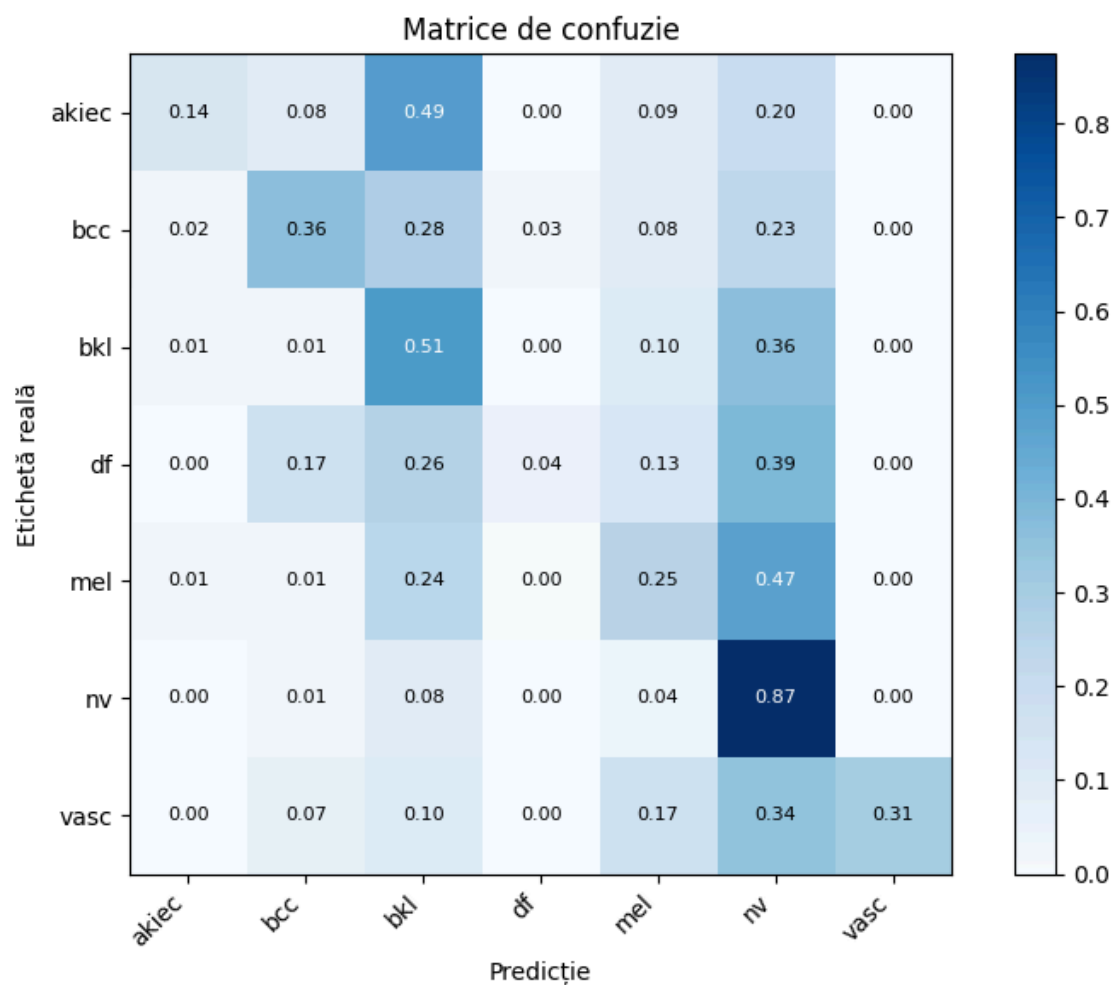
La fel ca mai sus, se observa ca datele acestui set sunt mai echilibrate.

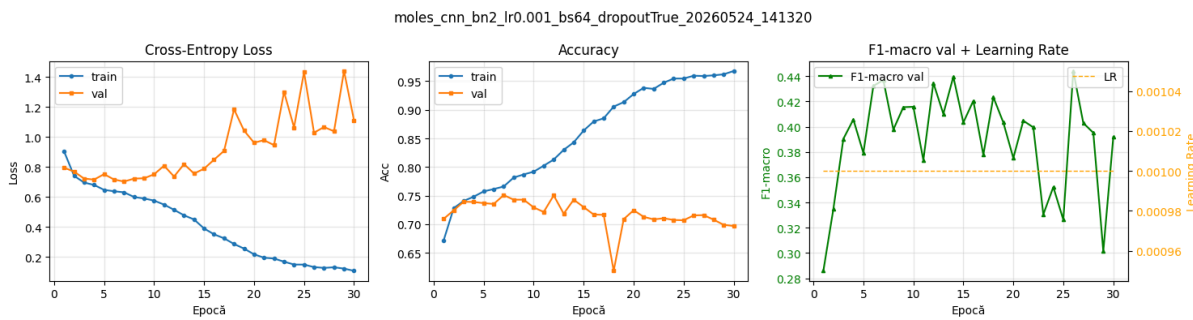
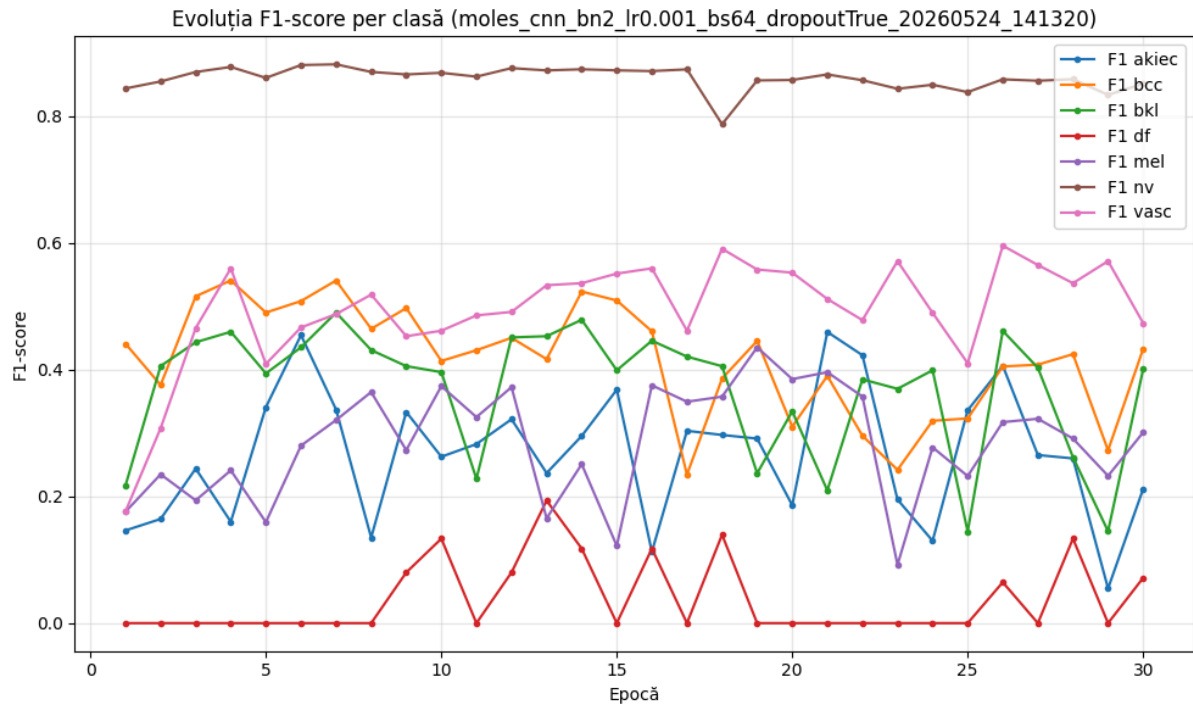


MOLES

CNN cu BatchNorm2d, Dropout marit de la 0.3 la 0.5 si weight_decay $1e-3$ de la $1e-4$:

Am rulat intai doar cu BatchNorm2d, dar overfitting-ul era in continuare foarte mare.

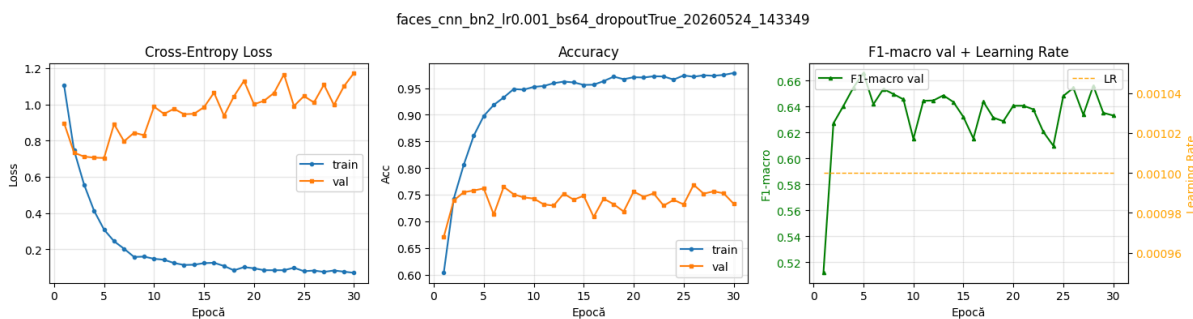
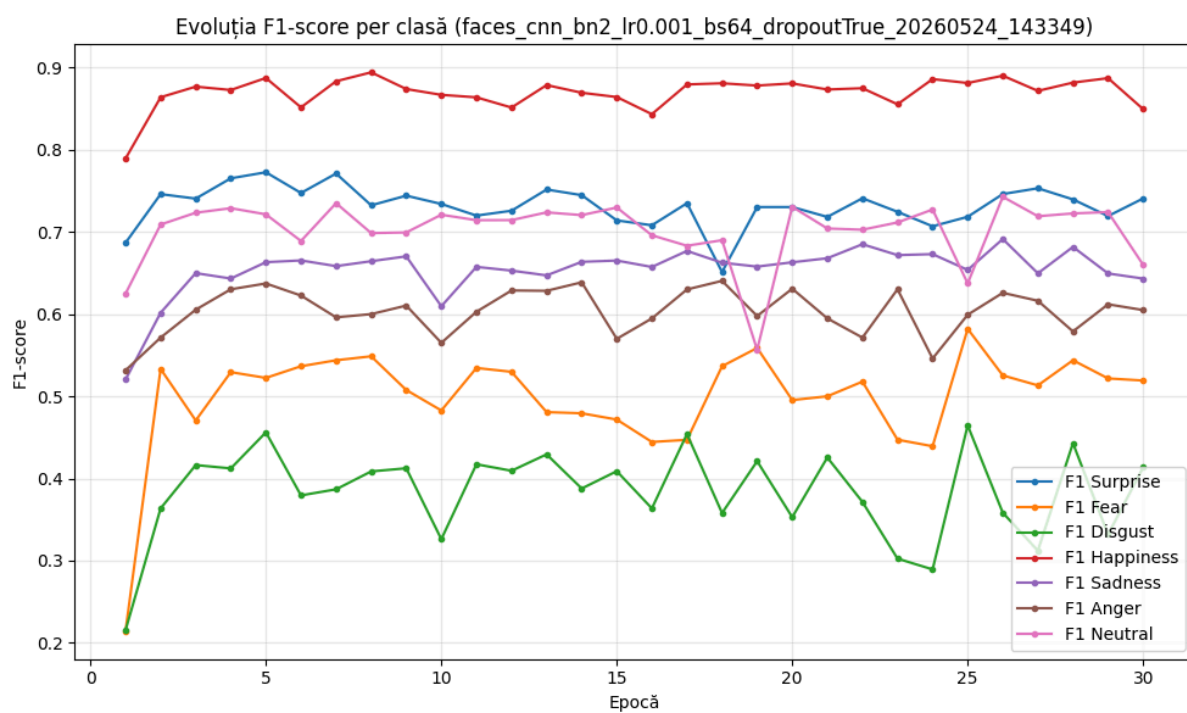
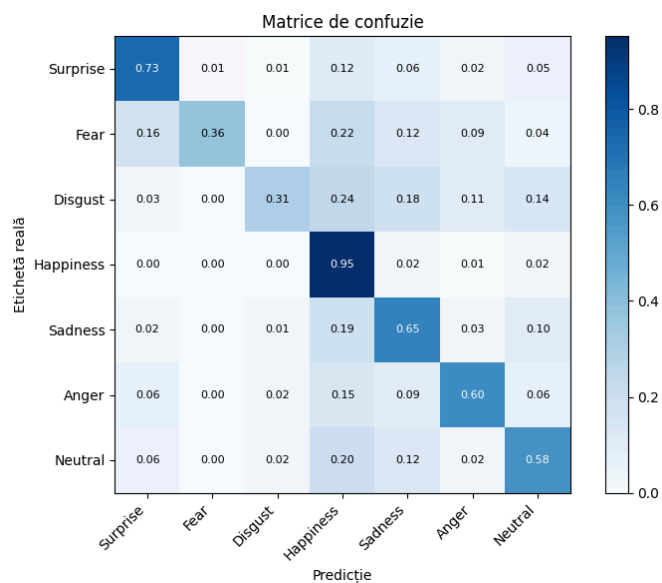




Nu se prea observa imbunatatiri.

FACES

CNN cu BatchNorm2d:



Se observa o crestere a F1-macro si a acuratetii, ambele cu aproape 5%.

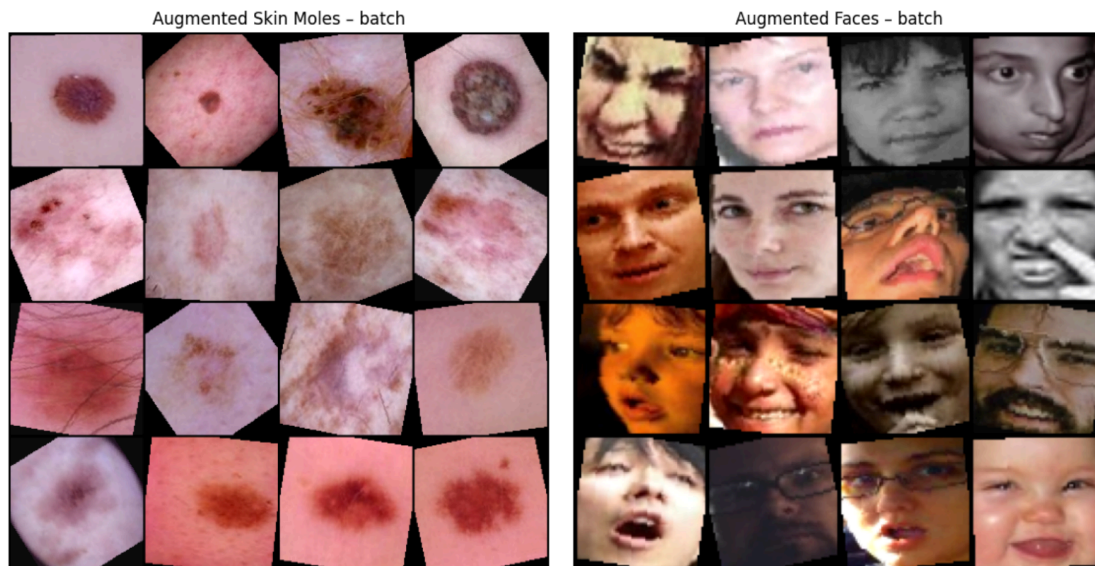
Augmentarea datelor:

```
moles_train_transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),          # OK - leziunea nu are
"sus/jos"
    transforms.RandomRotation(degrees=180), # OK - orice unghi e valid
    transforms.ColorJitter(brightness=0.1, contrast=0.1), # subtil! nu
distorsiona culoarea
    transforms.ToTensor(),
    transforms.Normalize(mean=MOLES_MEAN, std=MOLES_STD),
])

moles_val_transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=MOLES_MEAN, std=MOLES_STD),
])
```

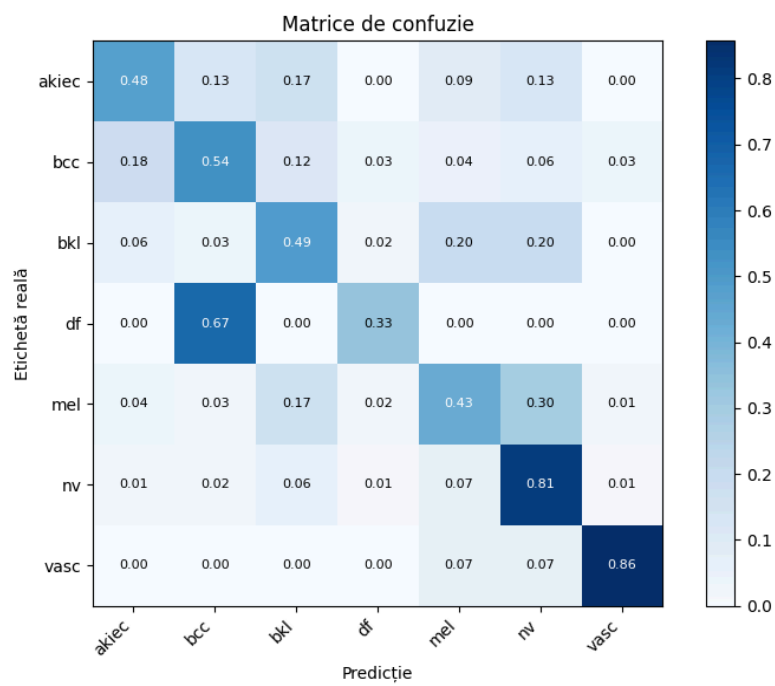
```
faces_train_transform = transforms.Compose([
    transforms.Resize((72, 72)),          # puțin mai mare ca să poată
crop
    transforms.RandomCrop(64),            # variabilitate de poziție
    transforms.RandomHorizontalFlip(),     # OK - fețe din ambele
direcții
    # NU RandomVerticalFlip - fețe cu capul în jos nu există în date
reale
    transforms.RandomRotation(degrees=15), # mic - față prea rotită nu
mai e naturală
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=FACE_MEAN, std=FACE_STD),
])

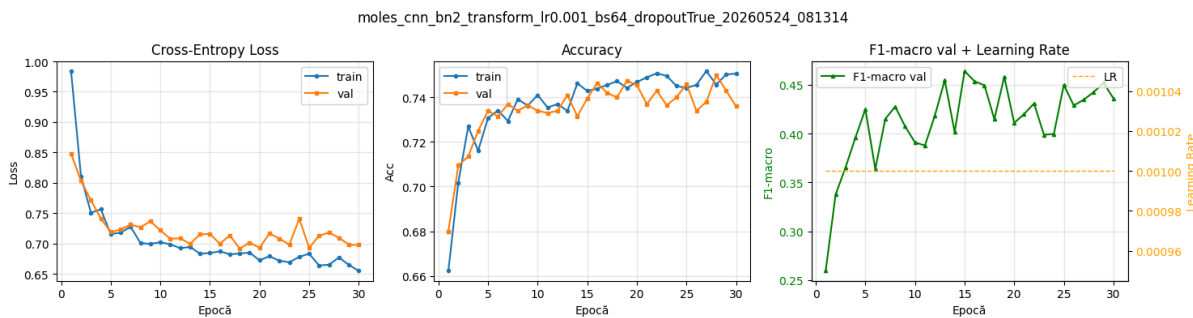
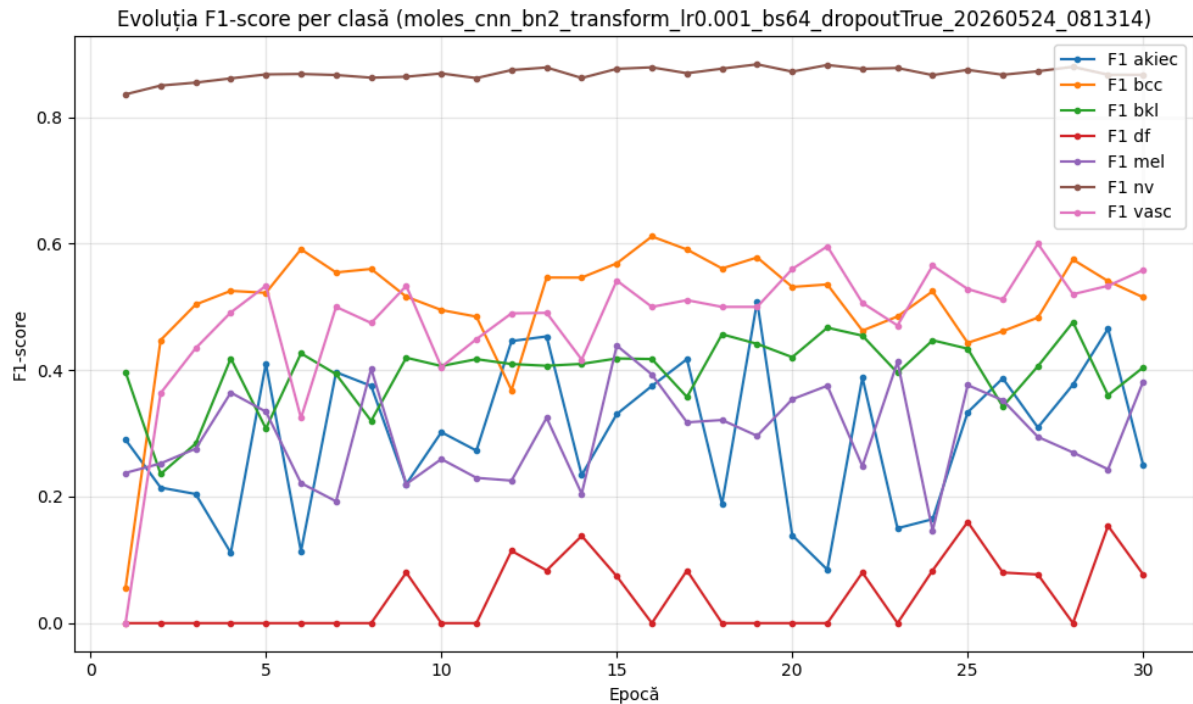
faces_val_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(mean=FACE_MEAN, std=FACE_STD),
])
```



MOLES

CNN cu BatchNorm2d si augmentare:

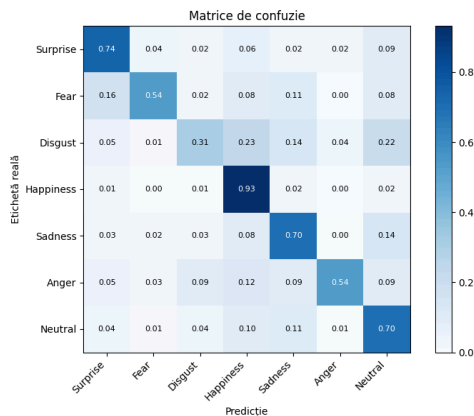


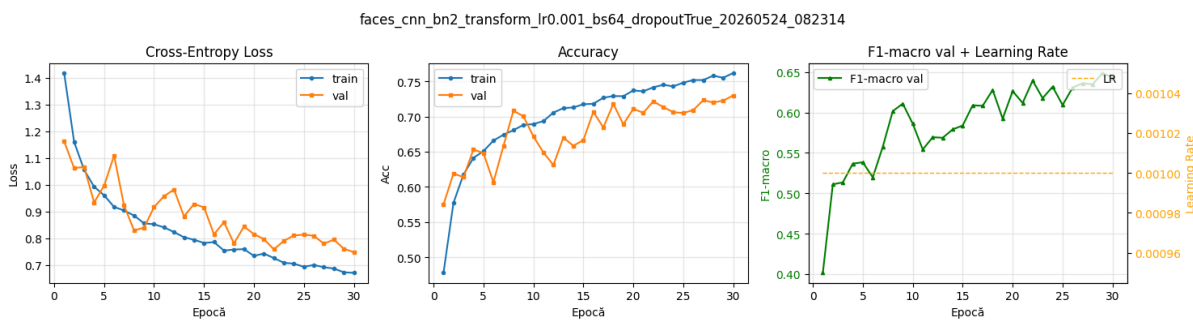
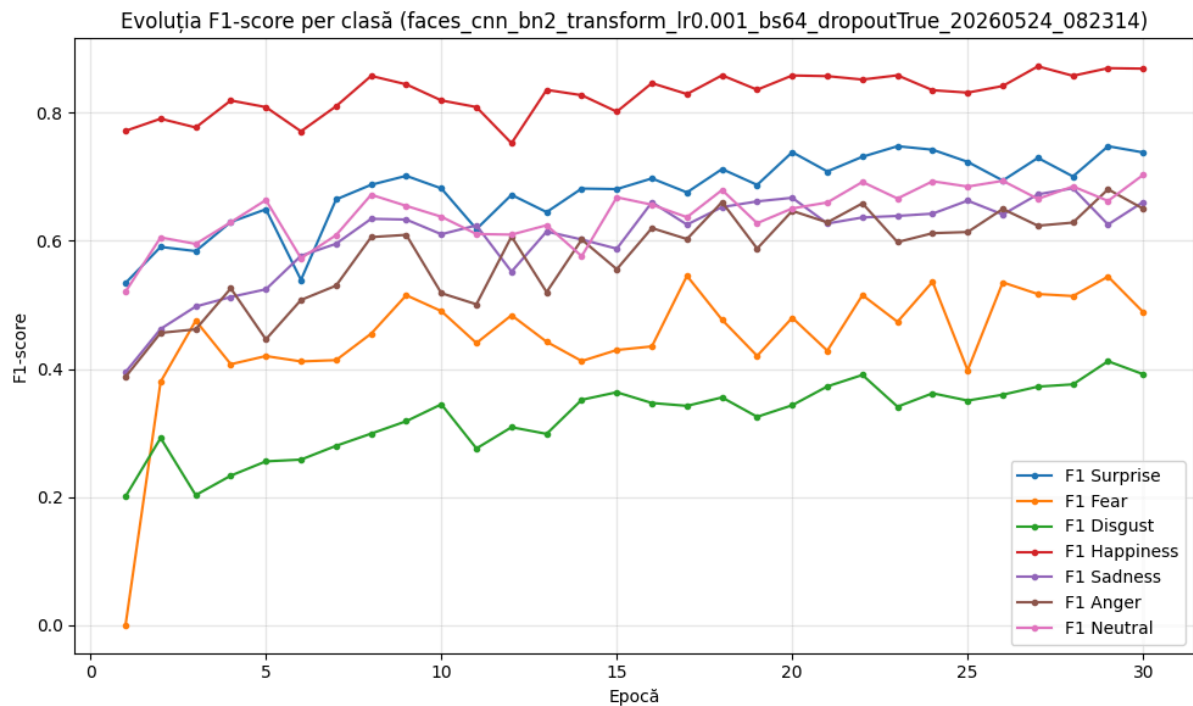


Se reduce diferența dintre antrenare și testare SEMNIFICATIV!

FACES

CNN cu BatchNorm2d și augmentare:

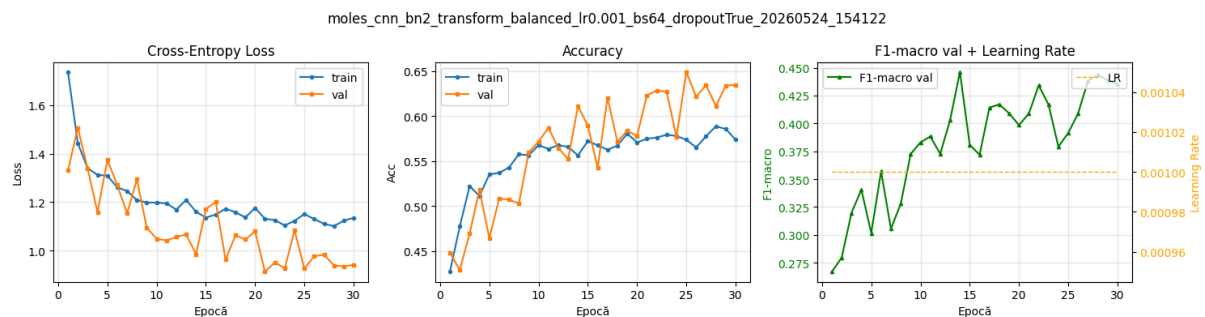
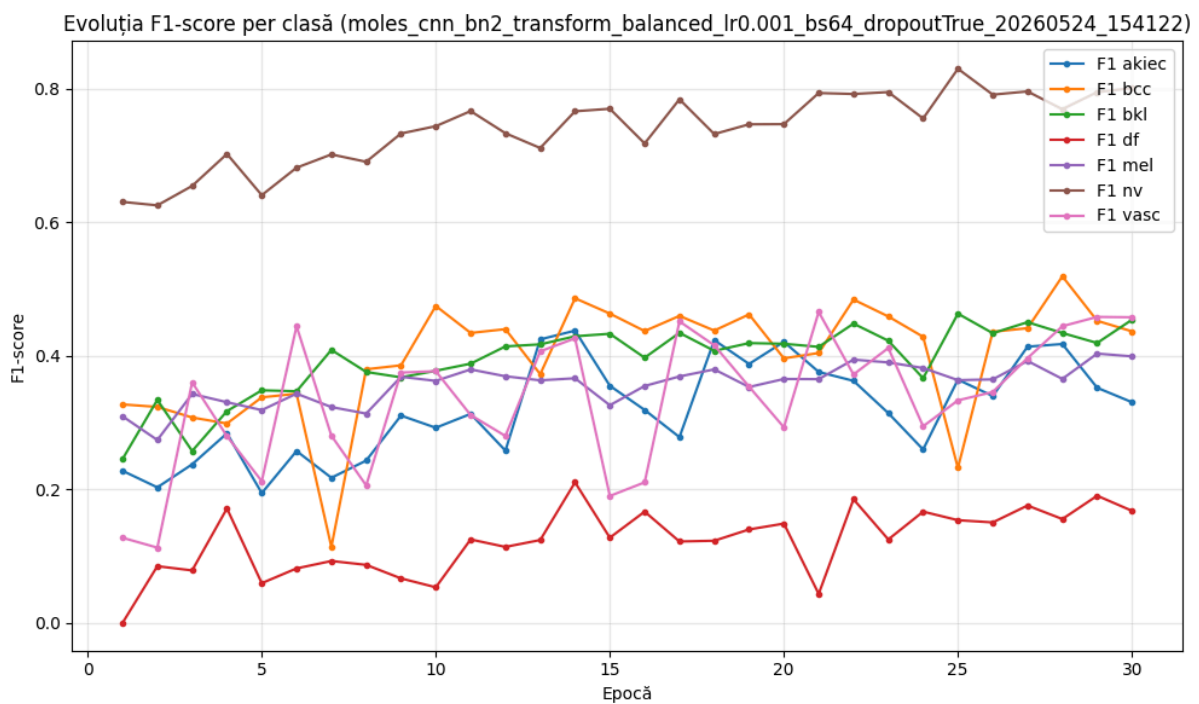
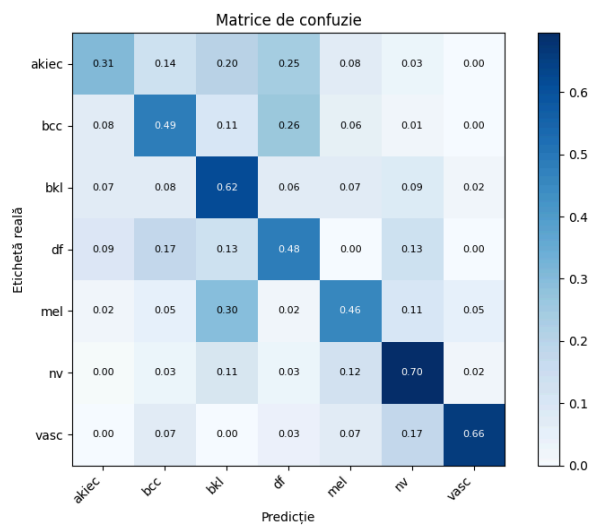




Se reduce diferența dintre antrenare și testare SEMNIFICATIV!

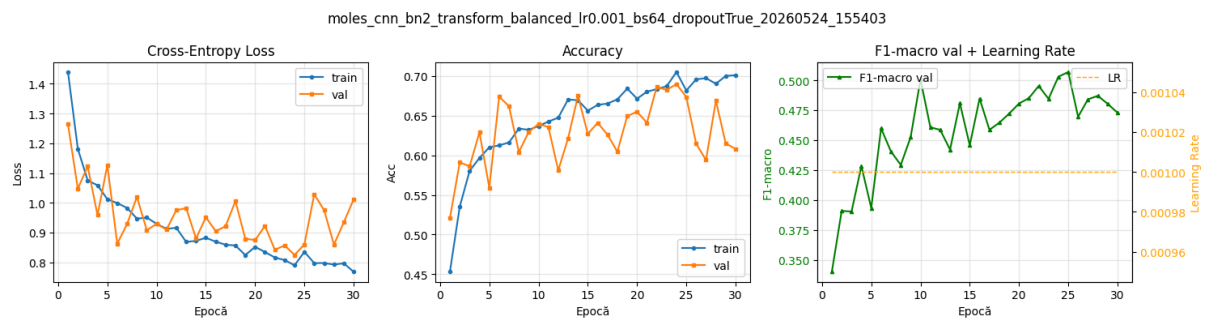
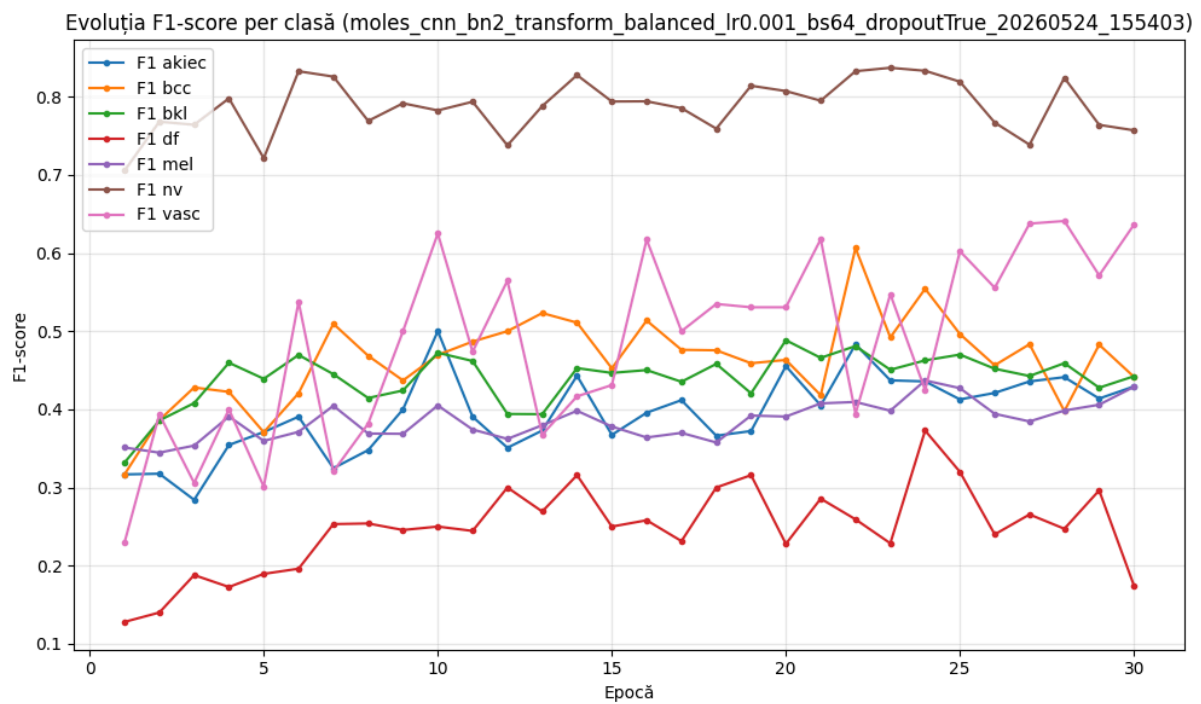
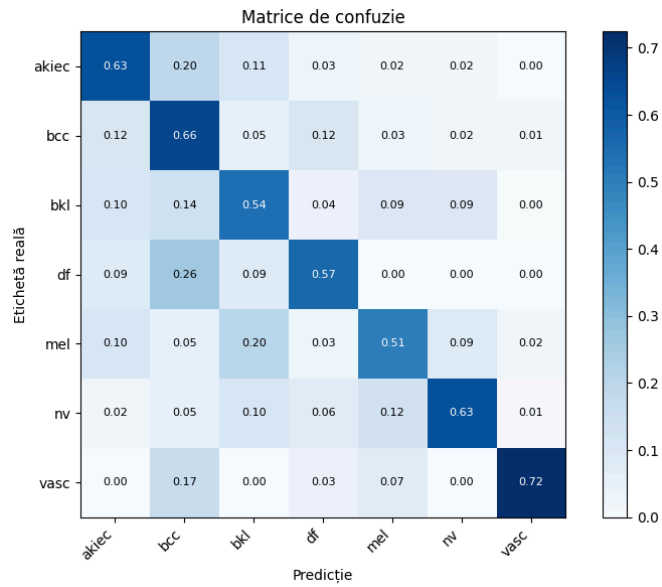
In continuare am încercat să rezolv dezechilibrul din clasele setului MOLES:

- Weighted Cross-Entropy-Loss - penalizări mai mari pentru gresirea claselor rare



Modelul de descurca mai bine pe teste decat in antrenare (underfitting)

- WeightedRandomSampler

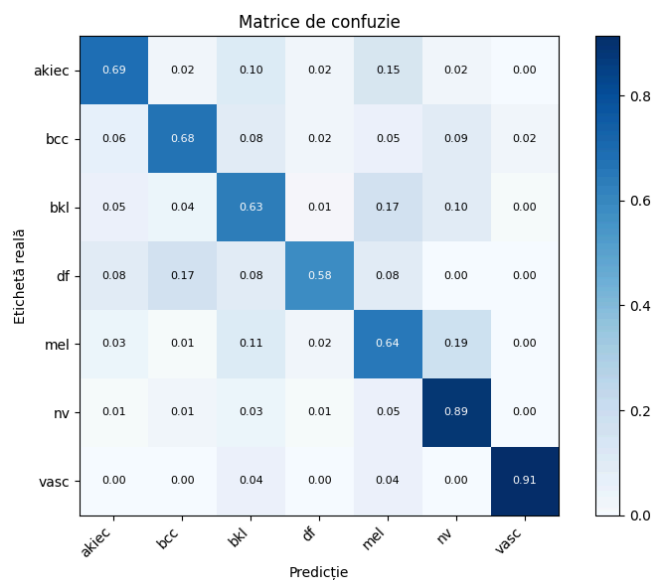


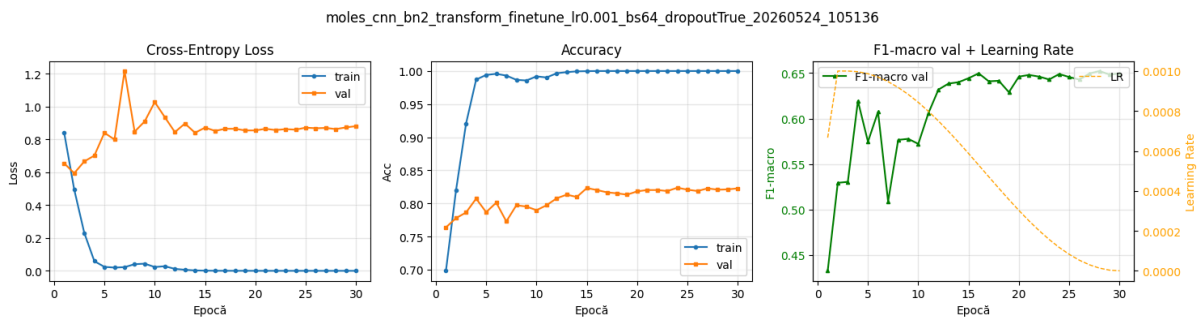
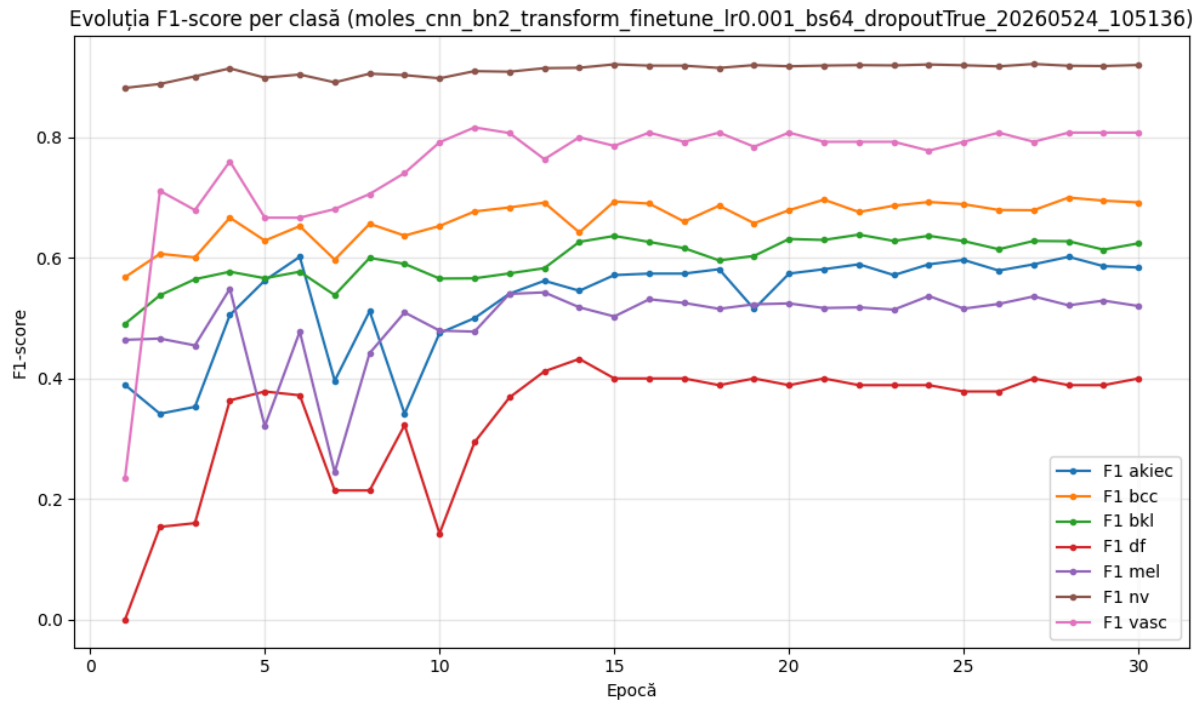
Cel mai bun rezultat de pana acum - acuratete: 65% - 70%,
F1-macro: 0.5.

Trecem la modele pre-antrenate:

Resnet18

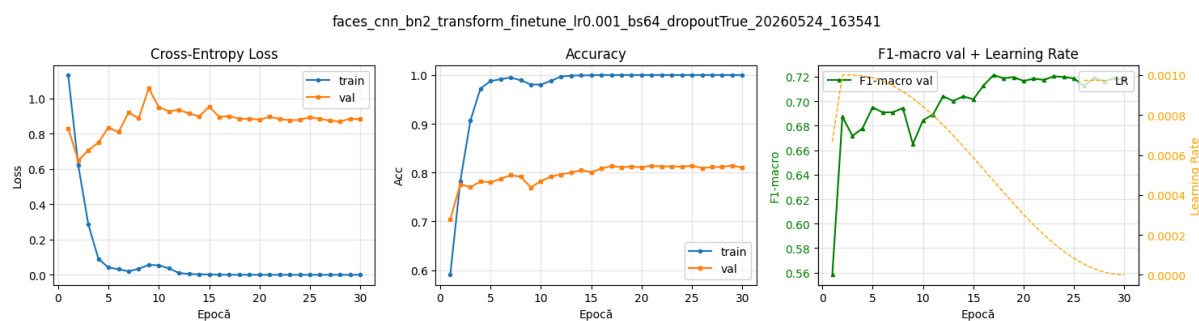
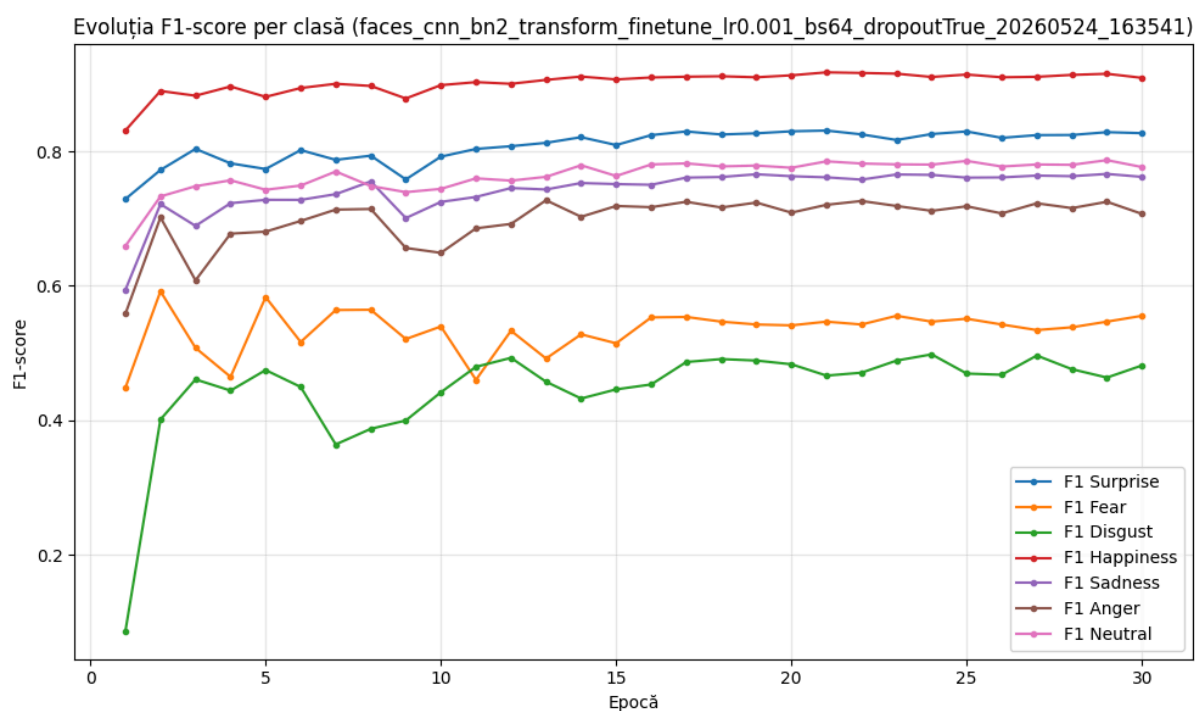
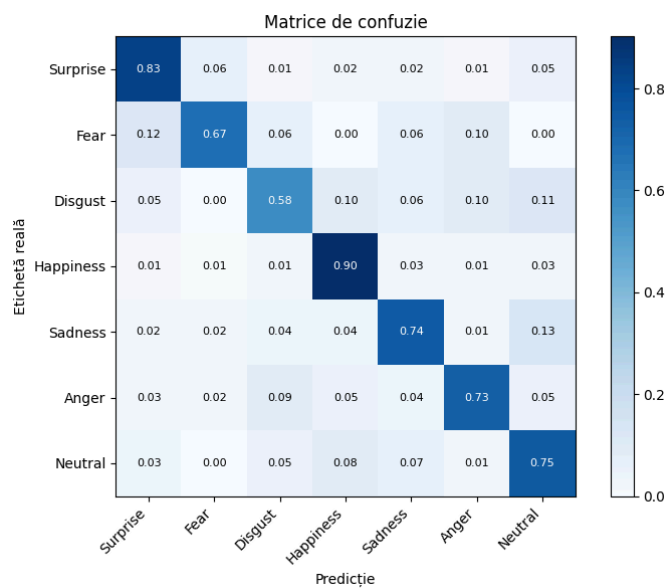
MOLES





Cel mai bun rezultat: accuratete: 80% si F1-macro: 65%

FACES



Cel mai bun rezultat: acuratete: 80% si F1-macro = 0.72